Can AI help keep data validation rules up to date?

Maintaining and extending expert rules when data standards change

MSc Artificial Intelligence thesis proposal

Tomash Oosterbaan
University of Groningen | Avanade and Rabobank

9 November 2026

Overview

What this thesis is about. Banks and listed companies send financial reports to supervisors in a fixed digital format. Every number in such a report carries a standard name, such as `Assets`, together with a date and a unit, so that software can read it. Before a report is accepted or used, software runs validation rules: small programs that check whether the numbers are consistent. A simple rule says that total assets must equal total liabilities plus equity. If a report states assets of $340 million and liabilities plus equity of $350 million, the rule raises an alert and an analyst investigates the difference (Figure 2 in Section 1). Supervisors and industry bodies publish hundreds of such rules, and banks add rules of their own.

Two problems. First, the dictionary of names behind the reports, called the reporting standard, changes every year. Names are replaced, removed or added, and report tables get new rows or columns. A rule that still refers to an old name can silently stop checking anything. For example, the 2026 US accounting dictionary replaced the name `BalanceSheetLocationAxis` by `StatementOfFinancialPositionLocationBalanceAxis`, and the committee that maintains the public US rules had to update every rule that used the old name. Today, experts find and update such rules by hand. Second, even up-to-date rules miss some errors. New rules can close these gaps, but a badly chosen rule can flood analysts with alerts on data that is correct.

What the thesis tests. Language models, AI systems that read text and write code, could help with both problems. The thesis tests whether they actually do, in three experiments (Figure 1):

1. **Updating rules (RQ1).** A model receives an old rule and the published description of what changed in the standard, and proposes the new version of the rule. The proposal is compared with the update that the experts actually published.
2. **Adding rules (RQ2).** A model proposes new rules. To test them, known errors are planted in copies of real reports, for example an amount multiplied by 1,000. We count how many planted errors the new rules catch that the existing rules miss, and how many extra alerts this costs.
3. **Testing over time (RQ3).** The updated and new rules are run on real past reports in the order in which they were filed, and on new reports as they arrive during the thesis. We count real errors found, meaning errors that companies later corrected and that were checked by hand, and the number of alerts raised.

Why this can be measured. Two organisations publish their rules together with every earlier version: the XBRL US Data Quality Committee (DQC), which writes rules for US company reports, and the European Banking Authority (EBA), which writes rules for the reports of European banks. The difference between the old and the new version of a rule is an answer written by experts. All data is public.

What comes out. The thesis delivers a public benchmark of real rule updates, measurements of when AI helps and when simple methods are enough, and a software platform in which every proposed rule is tested and first runs in shadow mode: its alerts are recorded but have no effect

on the data. A person then decides whether the rule should only be monitored, should warn, or may block data. Table 1 explains the terms used throughout. Each section is written so that it can be read on its own with this table at hand.

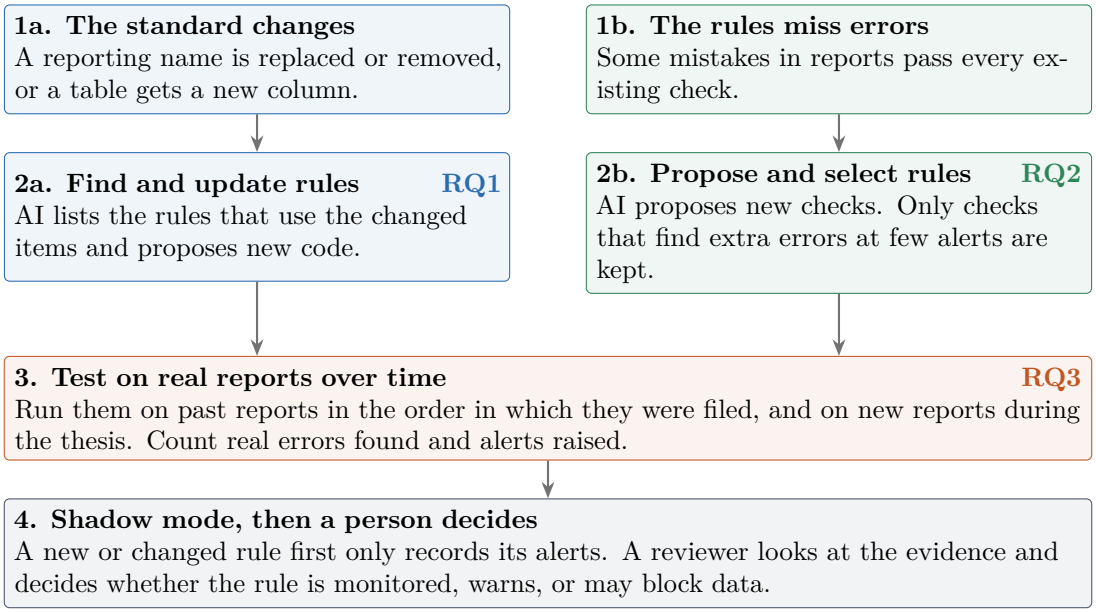


Figure 1: The workflow studied in this thesis. Each research question has an answer key that nobody in the project wrote. RQ1: the update that the experts published. RQ2: errors deliberately planted in copies of reports, at known locations. RQ3: real corrections that companies made later, each reviewed by hand.

1 Problem and research objective

Why validation matters. Data used for financial and supervisory reporting must be accurate, complete and consistent. The Basel Committee's BCBS 239 sets out principles for risk data aggregation and reporting, and the European Central Bank's 2024 guide describes the governance, controls and validation it expects from supervised banks [1, 2]. The ECB's 2024 supervisory report found continuing weaknesses in data accuracy, data architecture and reporting processes [3]. Validation rules are one way to meet these requirements. At Rabobank, technical validation, business rules, monitoring and the manual investigation of exceptions are among the controls in use [4].

What a validation rule does. Figure 2 shows a real rule. DQC rule `DQC_0004` reads two numbers from a report, total assets and total liabilities plus equity, and checks that they are equal when they refer to the same company, date and unit. The DQC documents the rule with an example in which assets are $340 million and liabilities plus equity $350 million. The rule then reports the $10 million difference so that an analyst can investigate [5]. Every rule has these three parts: the facts it reads, the condition it tests and the message it produces. The EBA publishes similar rules for European banks. These refer to numbered cells in standard report tables instead of named facts (Table 2).

How a change to the standard affects a rule. Rules refer to the names, tables, rows and columns defined in a reporting standard. For US company reports, the FASB publishes a new version of the US GAAP taxonomy every year, together with files that list which names are deprecated and which names replace them [7, 8]. For European banks, the EBA publishes successive releases of its reporting framework, with report tables, data definitions and validation rules [9, 10]. When a new version appears, a rule may need one of the four kinds of change in Table 3. The first two can be read from the change files of the standard. The last two require an

Term	Meaning in this proposal
Reporting standard	The dictionary of names that reports must use, with their meaning. For US company reports this is the US GAAP taxonomy (US GAAP are the US accounting rules; a taxonomy is a dictionary of reporting names), published every year by the Financial Accounting Standards Board (FASB). Each yearly version is a taxonomy year. For European banks it is the EBA reporting framework: numbered report tables with rows and columns.
Deprecated name	A name that a new version of the standard marks as outdated, often with a named successor. It is usually removed in a later version.
Change files	Files published with each US taxonomy year that list deprecated and removed names and, where one exists, the name that replaces them. A change notice is a short summary of these files that is given to a model.
Report, filing	A set of numbers that a company or bank submits to a supervisor. Each number is a fact: a name from the standard, a value, a date or period, and a unit.
XBRL	The digital format in which each number in a report is labelled with its name, date and unit.
Validation rule	A small program that checks a report. When the check fails, the rule raises an alert. Each DQC rule is one file that contains one or more checks, called sub-rules.
DQC	XBRL US Data Quality Committee. Publishes about 180 rules for US company reports, with their code and official test cases.
EBA	European Banking Authority. Publishes the report tables and validation rules for European banks.
Alert, alert budget	An alert is a value flagged for a person to review. The alert budget is the number of alerts per report that a team is willing to review.
Planted error	An error deliberately introduced into a copy of a report, at a recorded location. Used to test whether a rule catches it. A copy without planted errors is a clean copy.
Amended report	A corrected version of a report that a company files later. A changed number may reveal an error in the original.
Shadow mode	A new rule runs on real data and its alerts are stored, but it does not stop or change any data.
Monitor, warn, block	The three action levels of a rule: record alerts only; let data through with a warning; hold data for review.
Language model	An AI system that reads text and writes text or code. Open-weight models can be downloaded and run on one's own computers, so no data leaves the organisation.
Arelle	Open-source software that runs XBRL validation rules on reports.

Table 1: Terms used throughout the proposal.

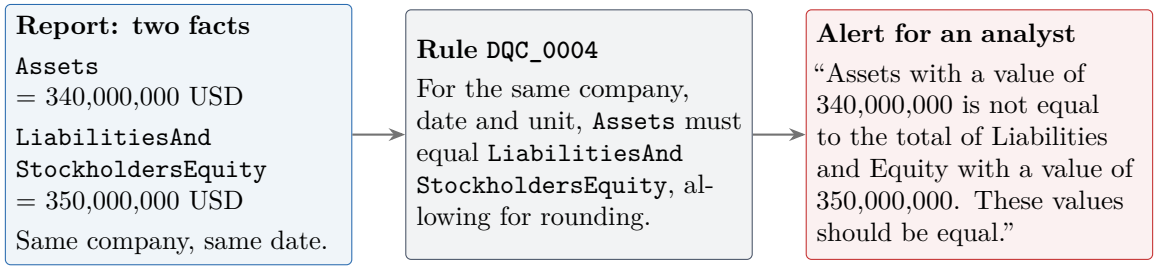


Figure 2: How a validation rule works. The numbers and the message text come from the DQC's documentation of rule `DQC_0004` [5], not from a real company.

understanding of what the rule is meant to check.

How often rules change. The 2026 DQC rule set for US reports contains about 180 rules, made up of 400 sub-rules. A pilot study for this proposal (Section 2.3) compared the DQC versions for the 2023, 2024, 2025 and 2026 taxonomies sub-rule by sub-rule. Over these three annual updates, the committee made 62 changes to the logic of sub-rules and added 56 new sub-rules. Changes that only renamed internal variables or edited the message text are not counted (Section 2.3). In the EBA rule workbooks, the EBA labelled 1,406 changes in 1,094 rules as changes to the formula

Publisher	What the data contains	What a rule checks (simplified)
DQC, for US company reports	Named financial figures such as assets, liabilities and equity, each with a date and a unit	`DQC_0004` checks that assets equal liabilities plus equity. The publisher's example compares $340m with $350m [5].
EBA, for European bank reports	Standard report tables in which each cell is identified by a table number, a row number and a column number	Rule `v6053_m` checks that one cell in table F 01.02 equals the sum of a set of cells in table F 09.01.1 [6] (Section 5.2).

Table 2: Two real kinds of validation rule. The DQC message and the EBA formula come from the publishers; the descriptions are simplified.

Kind of change	Real example	What the rule needs
A name is replaced by a named successor	The 2026 US taxonomy replaced `BalanceSheetLocationAxis` by `StatementOfFinancialPosition LocationBalanceAxis`. DQC sub-rule `US.0164.10087` used the old name [11].	Replace the name.
A name is removed without a successor	The 2025 US taxonomy removed `CashAndCashEquivalentsPeriodIncreaseDecrease ExcludingExchangeRateEffect`. DQC sub-rule `US.0004.9286` referred to it [11].	Remove the reference without breaking the rule.
A report table changes	EBA release 3.0 added column 0065 to the cells summed by rule `v6053_m`; the EBA's note reads "Added column on POCI", for exposures that were already credit-impaired when purchased or originated [9].	Change the formula.
The experts correct their own rule	In a hand-checked pilot sample of 12 DQC changes, three fixed the rule itself: a missing exception, a missing part of a calculation and a bracket error [11].	Correct the logic.

Table 3: Four ways in which a new version of a standard can require a rule to change. All examples are from the published DQC and EBA rule versions examined in the pilot work (Section 2.3).

itself; 266 of these only switched from three-digit to four-digit row and column codes. Formula edits under other labels, such as notation-only updates, come on top of this. Each change is a maintenance task with a published expert answer. These tasks are the subject of RQ1.

What happens to an old rule. This depends on the publisher. The DQC publishes a new version of its rules for each taxonomy year. The EBA distributes revised rule packages, including quarterly packages for corrections, deactivations and changes in severity [12, 9]. An organisation that has written its own rules must itself work out which rules a change to its data definitions affects. The platform proposed here lists the rules that are likely to be affected, suggests a change and tests the result.

The second problem: missing rules. A set of correct rules can still miss errors, and a data team usually cannot see the mistakes that pass all its checks. It can inspect past corrections, review incidents, read the reporting definitions and test its controls by planting errors in copies of the data. Language models can use these materials to suggest new rules, but a plausible rule can still be wrong. A proposed upper limit on a loan amount, for example, means nothing unless the business has set that limit. Constraints that are discovered automatically are often false, and code written by language models can differ from one run to the next [13, 14]. In financial reporting this matters, because an unjustified rule can produce many alerts on correct data. The research

therefore measures what each new rule adds to the controls that are already in use, and at what cost in alerts.

Objective. The objective is to establish when AI helps to maintain and extend expert validation rules, and how a data team can test each proposed rule before relying on it. The experiments use independently published rule changes, planted errors and historical reports. The resulting procedure is implemented in a platform with a review step and a quality gate in a data pipeline: a step that lets a batch of data through, lets it through with a warning, or holds it for review.

2 What the pilot studies showed

Four small pilot studies between September and November 2026 shaped the research questions. This section only describes what they contributed to the research. Working reports, code and data are in the public pilot repository [18].

2.1 From error detectors to expert rules

The first pilot tested automatic error detectors on three public tables. The best detector found 11 to 21 of 40 errors that simple rules missed, but it also flagged 520 to 900 correct values per 800 records, so a reviewer with limited time gained little [16]. Two lessons followed: an extra check is only useful if it raises few alerts, and the test data must come with real expert rules. The second pilot therefore used US company reports and the DQC's expert rules for them.

In that pilot, two small language models, Qwen2.5-Coder-7B and Qwen3-8B [21, 22], wrote rules. When they rewrote existing DQC rules from their description, their code raised about 41% of the alerts of the real rules for 10 rules published before the models were released, but only 3% and 13% for 6 rules published later. This suggests that the models had seen the older rules during training, so the thesis has to control for this. When the models proposed new rules, the existing rules caught 27 of 83 planted errors and the best set of new rules caught 6 more. That set, however, raised 458 alerts per report, against about 1.8 for the DQC rules [15]. Keeping only the new rules that added detections at no more than one alert per report left a single rule, which caught 3 of 195 missed errors at 0.21 alerts per report [18]. This is the starting point for RQ2: a new rule has to be judged by what it adds to the existing rules at a fixed number of alerts.

2.2 Corrected reports and testing over time

Companies sometimes file a corrected version of a report, called an amended report. Comparing an original report with its amended version shows which numbers changed. At first sight, every changed number looks like an error in the original report, but this is often not the case. In the pilot, 20 changed numbers were checked one by one against the original and amended documents and the companies' own explanations [15, 18]. In 9 cases the original number was wrong. In 7 of these, however, the company had later corrected its own bookkeeping (a restatement): the original report showed what was in the books at the time, so the mistake was not visible in the report itself. Only 2 were mistakes in the report itself: one number was filed under the wrong accounting name, and one amount for a single quarter was labelled as covering a whole year. In 3 cases the original number was not wrong, and in 8 the documents did not make clear whether it was.

A validation rule can only catch mistakes in the report itself. A changed number is therefore not automatically an error that a rule should find, and the dataset of real errors in this thesis reviews every change individually (D4, Section 5.4).

A second test ran the DQC rules on 22 reports filed between 2023 and 2025. Of the alerts, 46%

came from rules that did not yet exist when the report was filed, and none of the 13 checked corrections in these reports was flagged by any rule [17]. A fair test over time must therefore use the rules that existed at the time, and real errors that a rule can catch will be rare. The same data shows that corrections arrive quickly: of 652 reports from 2023 that were later amended, about half were amended within 90 days and 95% within a year [18]. These findings shape RQ3.

2.3 Rules that change with the standard

The third pilot compared the DQC rule versions for the 2023 to 2026 US taxonomies. Over three annual updates, the committee made 62 changes to the logic of its checks and added 56 new checks. In 28 of the 62 changes, the old rule used a name that the new taxonomy had replaced or removed, so the published change files explain the change. The other 34 had other causes, such as a corrected calculation [11].

Two simple tests followed. A search without AI flagged every rule that used a replaced or removed name: 23 of the 42 flagged rules had indeed been changed, but the search found only 23 of the 57 rules that changed. For 16 explained changes with short rule code, a find-and-replace script wrote 10 of the updates correctly and a small model, Qwen2.5-Coder-7B, wrote 14 [11]. These are the easy cases. RQ1 therefore measures where AI adds value beyond such simple methods, in particular on the harder changes and on changes in EBA tables.

3 Related research and scientific contribution

Earlier research has used language models to write data-quality rules, and software research has studied how code can be updated when its environment changes. No earlier study evaluates rule updates against the updates that experts published, measures what generated rules add beyond the expert rules a team already has, and tests both on historical data in time order.

Research on data cleaning has shown that different error detectors find different errors, and that every extra alert adds review work [23, 24]. This supports judging new checks by what they add within a fixed review budget.

Several studies let language models write data-quality rules. Alexander et al. studied the quality and consistency of validation tests written by language models, Akella et al. generated rules and code with retrieved domain knowledge, and LeDQeR generated and filtered rules for the Great Expectations library [25, 26, 27]. AuDQ combines language-model agents with rule discovery, documentation and human review [28]. Data-quality tools now offer such features commercially, and at least one such system has been patented [29, 30]. SPADE selects a small set of checks while limiting false alarms, although its checks concern the output of language-model applications [31]. None of these studies compares generated rules with the expert rules a team already has, or updates rules when a standard changes.

Software research has studied a related problem: code that must be updated after a library it depends on changes. CodeUpdateArena and GitChameleon test whether generated code still works after such a change, and SWE-bench uses real issues and fixes from software projects [32, 33, 34]. Continuous data cleaning studies data and constraints that change together [35], and Auto-Validate-by-History learns validation checks from earlier runs of a data pipeline [36]. These works motivate the use of real historical versions, executable tests and later data in this thesis.

The thesis contributes:

1. a public benchmark of real maintenance tasks for financial validation rules, each with the old rule, the published change to the standard and the experts' updated rule, on which AI and non-AI methods for finding and updating rules are compared;

2. a comparison of rules generated from reporting definitions alone with rules generated after the model has seen errors that the existing rules missed, together with a selection method that keeps only rules that add detections beyond the existing expert rules within a fixed alert budget;
3. a test of updated and new rules on real reports in filing order, which separates what was possible at the time from what is possible today, and which tests whether staged approval (monitor, warn, block) leads to decisions that still hold on later data;
4. a hand-reviewed set of real errors in SEC reports.

4 Research questions

The three research questions follow the workflow in Figure 1: keeping existing rules up to date, adding new rules, and checking both over time.

Main research question. Can open-weight language models help a data team maintain and extend validation rules, and what evidence is needed before a new or changed rule may block data, warn, or only be monitored?

RQ1. When a reporting standard changes, how well can language models find the expert rules that are affected and update them, compared with the updates that the experts published and with simple methods that do not use AI?

RQ2. How many additional errors do AI-generated rules detect beyond the existing expert rules when they are proposed from reporting definitions and example reports alone, and when the model also sees examples of errors that the existing rules missed? How do five methods for selecting among these rules compare at a fixed number of alerts per report?

RQ3. When updated and new rules are run on real reports in the order in which they were filed, and on new reports during the thesis, how many reviewed real errors do they detect, how many alerts do they raise, and do decisions to monitor, warn or block that were taken on earlier reports remain justified on later ones?

	In plain terms	Answer key	Main measures
RQ1	The standard changed. Does AI find the rules that must change, and write the right new version?	The updated rule that the experts published, and the publisher's test reports	Share of flagged rules that really changed; share of changed rules that were flagged; share of updates equal to the expert version or passing its tests
RQ2	Can AI add rules that catch errors the existing rules miss, without too many alerts?	Errors planted at known locations in copies of reports	Extra planted errors caught at 0.5, 1 and 2 alerts per report
RQ3	Do the rules hold up on real reports over time, and are approval decisions still right later?	Real corrections, each reviewed by hand; a hand-checked sample of alerts	Real errors caught; alerts per report; share of approval decisions that still hold

Table 4: The research questions in plain terms. Section 6 describes how each is tested.

5 Datasets and reference material

Each research question needs two things: data on which rules can be run, and an answer key that nobody in this project wrote, so that a result can be marked right or wrong. Four public sources provide this, labelled D1 to D4. The rule-change benchmark for RQ1 is assembled from D1 and D2. Table 5 gives an overview, and the subsections describe each source with an example.

Dataset	What it is	What we do with it	Answer key	RQ
D1 US company reports (Section 5.1)	Quarterly and annual reports of US listed companies, in which every number is labelled, e.g. `Assets` = 340,000,000 USD. Also the DQC's expert rules for these reports, one version per year, and the yearly change files of the US dictionary.	RQ1: give a method the 2025 rule and the 2026 change files, and ask for the 2026 rule. RQ2: let a model propose new rules, plant errors in copies of reports and count what the rules catch. RQ3: run the rules on real reports in the order in which they were filed.	RQ1: the DQC's own 2026 rule. RQ2: the planted errors. RQ3: dataset D4.	1–3
D2 European bank reports (Section 5.2)	The EBA's report tables, in which each cell has a table, row and column number, and the EBA's validation formulas in every release.	Give a method an old formula and the change to the tables, and ask for the new formula.	The formula the EBA published in the next release.	1
D3 European company reports (Section 5.3), extension	Annual reports of EU listed companies, labelled with IFRS names, and a public set of IFRS data-quality rules.	Repeat the RQ2 and RQ3 experiments on a second kind of report.	Planted errors; the published IFRS rules.	2, 3
D4 Real SEC corrections (Section 5.4), built during the thesis	Numbers that companies changed in a corrected report, each checked by hand (Figure 4).	Check whether the rules would have caught real errors, and how many alerts they raise on reports that were never corrected.	The hand-made label: a real reporting error or not.	3
Benchmark built from D1 and D2 (Section 5.5)	A list of real rule changes. Each task has an old rule, the published change and the experts' new rule (Figure 5).	Run every method for finding and updating rules on the same tasks and compare the results.	The experts' new rule.	1

Table 5: The datasets: what each one is, what we do with it, and what tells us whether a result is right.

5.1 D1: US company reports and DQC rules

The SEC receives quarterly and annual reports from listed US companies. Every number in them is labelled with an accounting name, a unit and a date (Figure 2). The DQC publishes about 180 rules for these reports. They are written in XULE, a language for checking such data, and the open-source program Arelle can run them. For each rule, the DQC also publishes small test reports that show what the rule must flag [12, 37]. The SEC's Financial Statement Data Sets also show which reports were later corrected: between 2023 and mid-2025, 1,294 of 68,875 reports were followed by an amended version [15, 20]. For the thesis, these data are extended to the most recent quarter available.

5.2 D2: EBA rules for European bank reporting

European banks report their financial position in standard tables, called templates. The EBA gives each template a code, such as F 01.02, and numbers its rows and columns. A validation rule refers to cells by template, row and column, and relates them in a formula [9, 10]. Figure 3 shows a real rule, `v6053_m`. In words: the provisions a bank holds for commitments and guarantees it has given, reported in template F 01.02, row 0220, column 0010, must equal the sum of the related amounts in the template for off-balance-sheet exposures, F 09.01.1 [6].

F 01.02 Balance sheet: liabilities

Row	0010
...	
0220 Commitments and guarantees given	**A**
...	

A must equal the sum of all cells marked **B** in template F 09.01.1 below. Rows marked ... are other rows of the templates that this rule does not use.

sum of all **B**

F 09.01.1 Off-balance-sheet exposures

Row	0040	0050	0060	0065	0110
0010 Loan commitments given	**B**	**B**	**B**	**B**	**B**
...					
0090 Financial guarantees given	**B**	**B**	**B**	**B**	**B**
...					
0170 Other commitments given	**B**	**B**	**B**	**B**	**B**

Orange: column 0065 was added to this rule in release 3.0. The EBA's note reads "Added column on POCI" (purchased or originated credit-impaired).

The same rule in two releases, as published by the EBA

Release 2.10: `{F 01.02, r220, c010} = xsum({F 09.01.1, (r010, r090, r170, c040, c050, c060, c110)})`

Release 3.0: `{F 01.02, r0220, c0010} = xsum({F 09.01.1, (r0010, r0090, r0170, c0040, c0050, c0060, c0065, c0110)})`

Two changes at once: all codes now have four digits (notation only), and column 0065 is added (a real change).

Figure 3: EBA rule `v6053_m`. The grids are simplified excerpts of the two templates; only the rows used by the rule are shown. Formulas and change note from the EBA rule workbooks [9]; row labels from the EBA's Q&A on this rule [6].

The EBA publishes its rules again with every release of its reporting framework, with a label for the kind of change. In the releases examined, the EBA labelled 1,406 changes in 1,094 rules as changes to the formula; 266 of these only changed the notation [11]. For RQ1, a method receives an old formula and the change to the templates, and proposes the new formula. The formula the EBA published in the next release is the answer key. The EBA does not publish test reports for every rule, so formulas are compared directly and differences are checked by hand. This dataset is the closest to the reporting work of a bank.

5.3 D3: European company reports and ESEF rules

Listed companies in the European Union publish their annual reports in the European Single Electronic Format (ESEF), with numbers labelled by IFRS accounting names [38]. XBRL International publishes a voluntary set of IFRS data-quality rules with test files [39, 40]. Rule `DQR_IFRS_0080`, for example, flags values that should not be negative, with messages such as "Accruals has a value of -400,000 which is less than zero" [41]. D3 is used to repeat RQ2 and RQ3 on a second kind of report. It also offers a translation task: a US rule that compares `Assets` with `LiabilitiesAndStockholdersEquity` has to compare `Assets` with `EquityAndLiabilities` when it is moved to IFRS names.

5.4 D4: a reviewed dataset of real SEC corrections

D4 is built during the thesis. When a company files an amended report, the two versions can be compared number by number. As the pilot showed (Section 2.2), a changed number is not necessarily an error that a rule could have found. Each change is therefore reviewed by hand, using the company's explanation and the context of both numbers. Figure 4 shows two real cases.

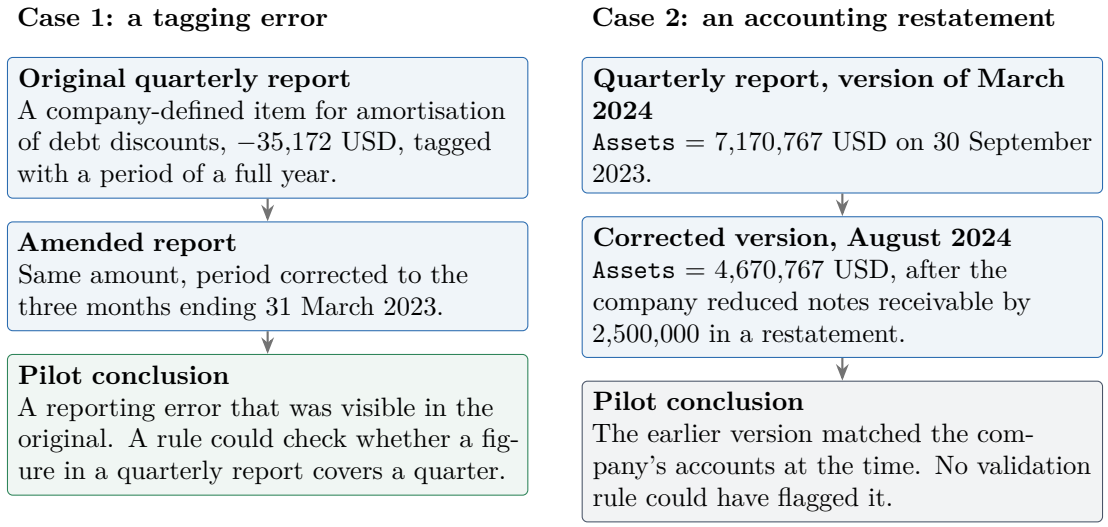


Figure 4: Two real correction cases from the pilot review, checked against the SEC documents [15, 18]. Company names are omitted.

The target is 300 changed numbers, spread over companies and kinds of change, with emphasis on changes that look like reporting mistakes: wrong signs, units, names or periods. For each item the dataset records the company, the filing dates, both values, where the evidence was found, whether the original was wrong, and whether a rule could have detected the problem from the original report alone. Each item is labelled directly from the source documents. A second reviewer independently checks 100 items, agreement is reported with Cohen's kappa [42], and cases that remain unclear are labelled as uncertain. Separately, 200 randomly chosen alerts on reports that were never corrected are checked by hand, to estimate how many alerts are false alarms. Because only 2 of 20 changes in the pilot were reporting mistakes that a rule might catch, D4 is expected to contain a few dozen such errors, not hundreds. RQ3 therefore reports its results on D4 with confidence intervals.

5.5 The rule-change benchmark for RQ1

The benchmark is not a new data source. It combines the DQC rule versions from D1 and the EBA formula versions from D2 into a list of maintenance tasks. Each task has an old rule, the published change to the standard and the experts' new rule, together with the dates on which each became available. A method sees only the old rule and the change; the experts' rule is the answer key (Figure 5).

Tasks differ in difficulty (Table 6). Some only need a name to be replaced. Others need a reference to be removed, or a calculation to be corrected in a way the change files do not describe. Every task is classified as a name change, a logic change or a notation-only change, and a second person checks a sample of the classifications before the benchmark is published.

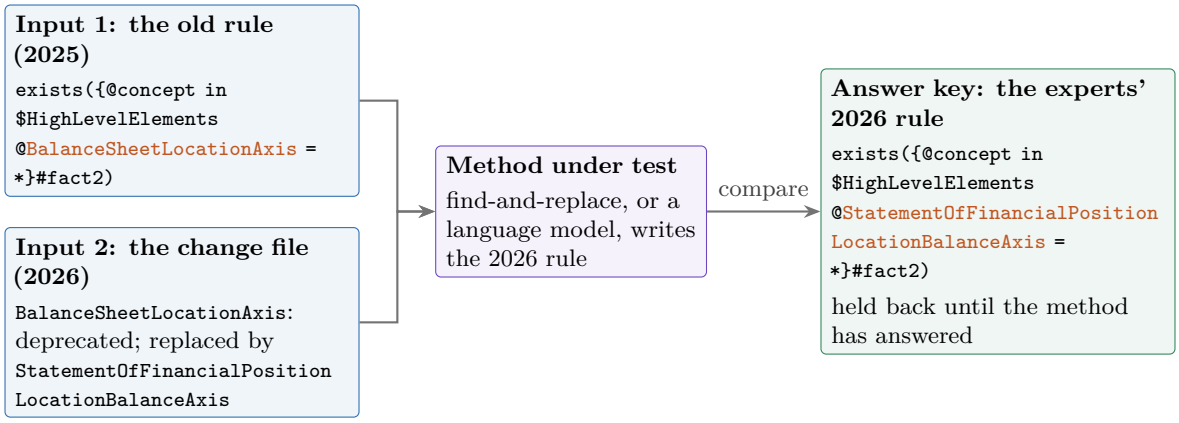


Figure 5: One rule-update task, from DQC rule `DQC_0164` (sub-rule 10087). The rule flags totals such as `Assets` that are split by balance-sheet location. In 2026 one name in it was replaced, and the experts updated the rule. A method sees the two inputs and must produce the new rule; the experts' version is the answer key. Only the coloured part differs.

Task	What changed	Find-and-replace	Small model
A. `DQC_0164`, 2025 to 2026	A name was replaced by a named successor (Figure 5).	Correct	Correct
B. `DQC_0004`, 2024 to 2025	A name was removed without a successor. The experts deleted it from a list of two alternatives.	Not possible	Correct
C. `DQC_0046`, 2023 to 2024	A name was deprecated without a named successor. The experts chose a different, longer name.	Wrong	Wrong

Table 6: Three real tasks from the third pilot and the results of find-and-replace and Qwen2.5-Coder-7B [11].

5.6 What counts as a correct result

In RQ1, the published expert update is the expected answer. A method may write different code that behaves the same way, so proposals are also run on the DQC's test reports where these exist, and differences are checked by hand on a sample. In RQ2, the answer key is the location of each planted error. Neither the models nor the selection methods see the planted errors of the final test. In RQ3, the answer key is D4 together with the hand-checked sample of alerts. In all three, alerts on reports that are not known to contain errors are counted as review work, because those reports are not proven to be correct.

6 Experimental design

This section explains how each research question is tested. Some rules apply to all three. Every AI method is compared with a simple method that does not use AI. All thresholds and alert budgets are fixed before the final tests. Development and test data are kept separate. Results are reported with 95% confidence intervals, including negative results.

6.1 RQ1: finding and updating rules after a change

RQ1 uses pairs of successive versions of a standard. For the DQC, the input is the 2025 rules, the 2025 taxonomy and the published changes for 2026; the 2026 rules stay hidden until the evaluation. The same is done for the earlier years and for successive EBA releases. The test has two steps (Figure 6).

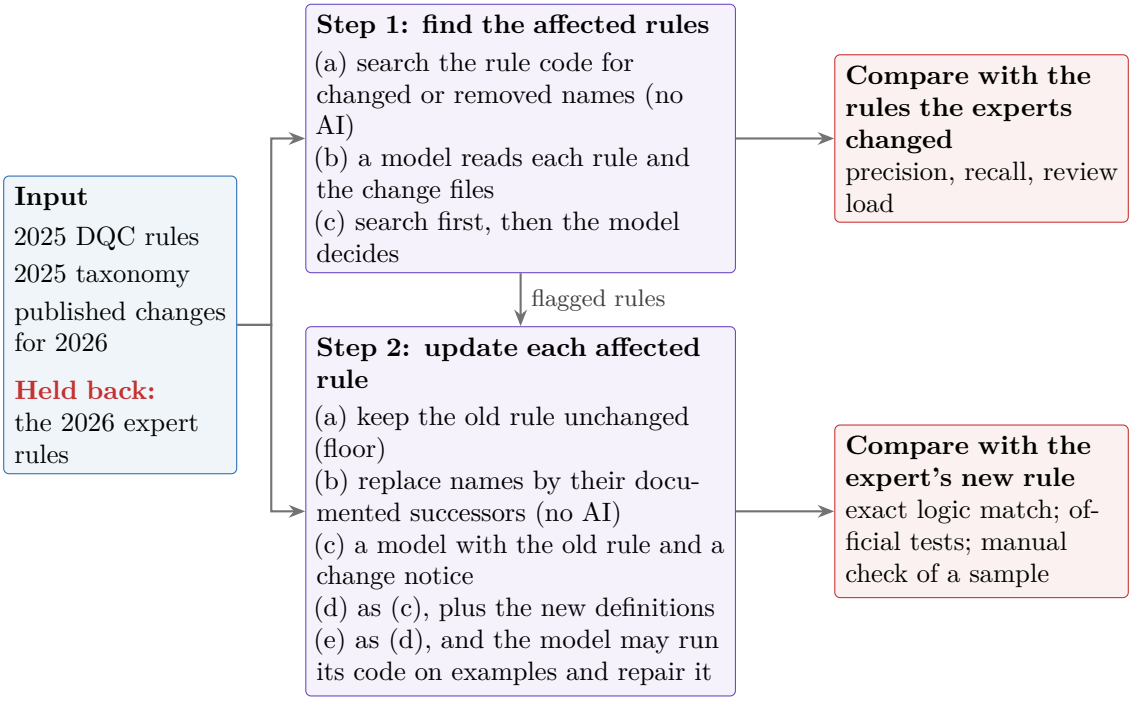


Figure 6: RQ1. Every AI method is compared with a method that uses no AI. Each model setup is repeated five times.

In the first step, methods decide which rules need attention. A simple search looks for names that the new standard replaces or removes. A language model reads each rule together with the change files. A third method lets the search propose candidates and the model judge them. We count how many of the rules that the experts changed are found, how many of the flagged rules were in fact changed, and how many rules a person would need to open.

In the second step, each affected rule is updated. Keeping the old rule and replacing old names with their documented successors are the baselines. The model gets three levels of help: the old rule and a short change note; the same plus the new definitions; and the same plus the option to run its code on examples and repair it. Each proposal is compared with the experts' new rule, and for the DQC it is also run on the official test reports. Each model setup is repeated five times, and results are reported separately for name changes, logic changes and notation-only changes.

Public rule versions may have been part of a model's training data. The drop from 41% to 3–13% in the second pilot suggests that this matters. The study therefore records when each rule version and each model became available, and compares results on rule versions published before and after the model's release. For OLMo 2, whose training data is public, it checks directly whether the rule texts occur in that data [43, 44].

6.2 RQ2: finding gaps and selecting new rules

RQ2 uses D1, with D3 as a second setting. The existing DQC or IFRS rules, together with simple statistical checks, are the baseline. The question is whether a model can propose reusable rules that catch errors the baseline misses, without raising too many alerts (Figure 7).

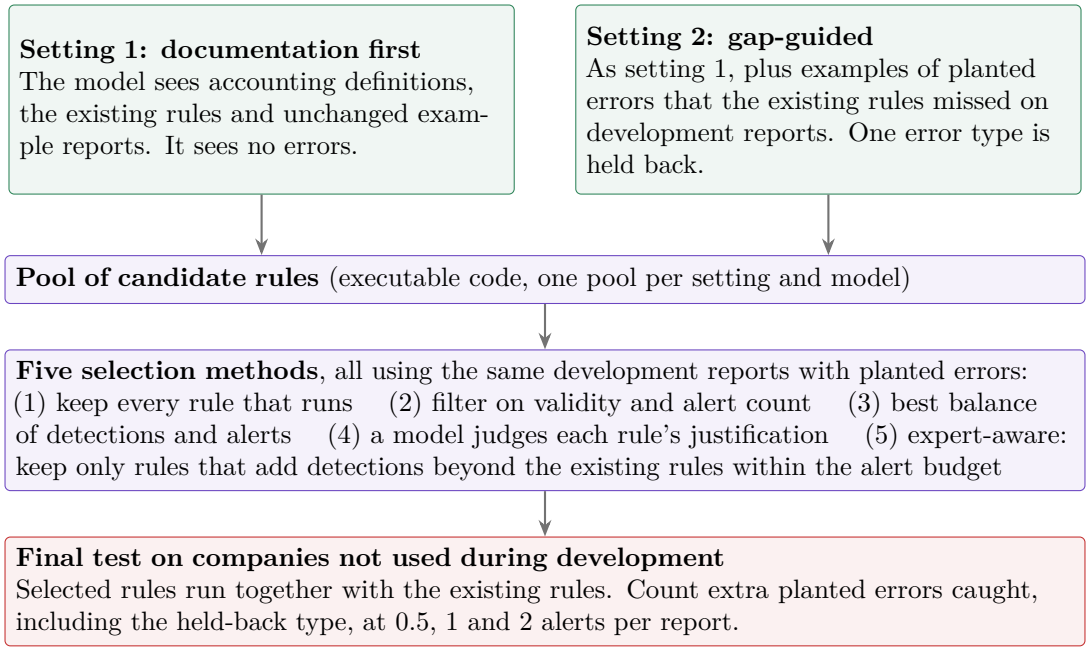


Figure 7: RQ2. The two settings differ only in what the model sees when it writes rules. Selection and testing are the same for both.

New rules are generated in two settings. In the first, the model only sees accounting definitions, the existing rules and example reports. It does not see any errors. This is the realistic situation, because a data team does not know which errors its rules miss. In the second setting, errors are first planted in copies of development reports: a reversed sign, an amount multiplied by 1,000, a mistyped digit, a wrong reporting period, a wrong accounting name or a missing number [19]. The existing rules are run on these copies, and the model sees examples of the planted errors they missed. In practice, past incidents and corrections could play this role.

Each setting produces a pool of candidate rules. Five methods then select from each pool, all using the same development reports. The first keeps every rule that runs. The second keeps rules that are valid and do not alert too often, as in LeDQeR [27]. The third picks a small set that balances detections and alerts, inspired by SPADE [31]. The fourth lets a model judge whether each rule is justified. The fifth, proposed in this thesis, starts from the existing rules and keeps only new rules that catch errors the existing rules miss, within the alert budget, and that do so for more than a few companies.

The final test uses reports from companies that were not used to develop or select the rules. The selected rules run together with the existing rules, and we count how many extra planted errors they catch at 0.5, 1 and 2 alerts per report. For comparison, the DQC rules raised about 1.8 alerts per report in the pilot. To avoid a circular test, one error type, the wrong reporting period, is never shown during development and only appears in the final test. RQ3 then tests the same rules on real corrections, which no error generator produced.

6.3 RQ3: testing the rules on real reports over time

RQ3 asks whether the rules from RQ1 and RQ2 still work on real reports, and whether a decision to let a rule monitor, warn or block would have been the right one. The test has four steps.

First, about 2,000 SEC reports filed between 2023 and late 2026 are collected and sorted by filing date. They include every report that contains an item of D4, plus a random sample of other reports.

Second, three sets of rules are run on every report: the DQC rules as they stood on the filing date, the updated rules from RQ1, and the new rules selected in RQ2. Every alert is stored with the report, the rule and the date.

Third, the alerts are scored. For each rule we count how many real errors from D4 it flags, how many alerts it raises per report, and which share of its alerts point to a real problem, based on the hand-checked sample of 200 alerts. A report that was never corrected is not assumed to be correct; its alerts count as review work.

Fourth, approval decisions are tested. Each new or updated rule starts in shadow mode: it runs on the reports of a first period, for example 2023 and 2024, but only records its alerts. At the end of that period, criteria that were fixed in advance decide whether the rule may monitor, warn or block. A rule may for instance only block data if it raises fewer than a set number of alerts per report and a set share of its checked alerts are real problems. The decision is then checked on the reports of the next period, for example 2025 and 2026. If the rule now raises far more alerts, or fewer of its alerts are real, the decision did not hold. To give an invented example: a rule that checks that revenue is never negative might raise 0.1 alerts per report in the first period, with 3 of 5 checked alerts being real errors, and be allowed to warn. If it raises 0.5 alerts per report in the second period and only 1 of 10 checked alerts is real, the decision did not hold. Staged approval of this kind is compared with putting each rule straight into its proposed level.

A fair test over time must not use knowledge that did not exist at the time. Three dates are therefore stored for every alert (Figure 8): when the rule version was published, when the language model that wrote the rule was released, and when a correction of the report became known. Expert rules are replayed as they were on each filing date. If older DQC rule packages cannot be run with current software, the current rules are filtered by their first release date, as in the pilot. Rules written by a model are only evaluated on reports filed after that model was released, because the model may have learned from anything published earlier; results on earlier reports are reported separately. Because 95% of corrections arrived within a year in the pilot data, reports filed in the last year before the corrections are collected are also reported separately.

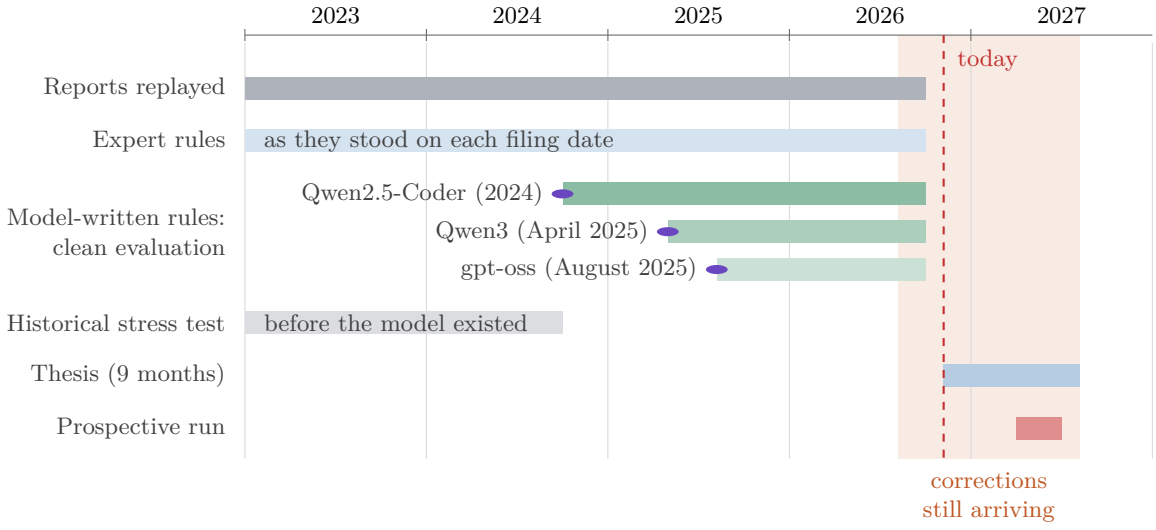


Figure 8: The time design of RQ3 (schematic; thesis months approximate). Expert rules can be replayed for the whole period. Rules written by a model are evaluated only on reports filed after the model's release; earlier results count as a historical stress test. The prospective run uses rules and thresholds that were frozen before any of its reports existed.

A prospective run tests the rules on reports that did not exist when the rules were fixed. At the end of month 5, the selected rules and the approval criteria are frozen and stored as a dated version in the public repository. They are then run every week on new SEC reports until month 8.

Because only about 2% of reports are later amended, this run is not expected to find many real errors. Its value is to show how frozen rules behave on unseen reports: whether they still run and how many alerts they raise.

Finally, RQ3 measures the cost of an outdated rule. Reports from one taxonomy year are checked with the previous year's rules, with the experts' updated rules and with the model's updated rules. The differences in alerts show what an outdated rule misses or adds, and whether the model's update behaves like the experts' update.

7 Models and baselines

The AI methods use open-weight language models: models that can be downloaded and run on the university's own computers, so that no data leaves the organisation and the exact model version and its release date are known (Table 7). Larger runs use the University of Groningen's Hábrók cluster [45]. The models only write text or code. Arelle runs the code on real reports, and a model's written answer never counts as a detection. Every AI method is compared with a method without AI: a name search and find-and-replace for RQ1, and the existing expert rules with simple statistical checks for RQ2.

Model	Released	Role
Qwen2.5-Coder 7B and 32B [21]	2024	Specialised in code; the 7B version continues the pilots.
Qwen3 8B and 32B [22]	April 2025	General model; the 8B version continues the pilots.
gpt-oss-20B [46]	August 2025	A second developer, to test whether results depend on one model family.
OLMo 2 7B and 32B [43]	2024–2025	Public training data, which allows a direct check of prior exposure.

Table 7: Planned models. "B" stands for billions of parameters, a rough measure of size. For the time design of RQ3, the release date is the date of the exact model version that is downloaded.

8 Engineering artefact

The procedure is built into a small software platform that a data team could use. It has four parts. A rule register stores every rule with its version, origin and approval status. A change check lists the rules that use changed names when a new version of a standard appears, and proposes updated code. A test bench plants errors in copies of data, runs candidate rules, replays them on past reports and shows the results to a reviewer. A quality gate in a small data pipeline, built with Apache Spark [47], lets data through, adds a warning or holds it, depending on the approved rules; new rules first run in shadow mode.

A person approves every rule. The platform never approves its own proposals, and generated code runs without access to production systems. The pipeline is demonstrated on public SEC data only. The scientific results come from RQ1 to RQ3; the platform shows how they could be used in practice, for example for locally written rules at a bank. The experiments do not measure Rabobank's own maintenance costs or error rates, and the platform would need internal security approval before any production use.

9 Plan, scope and expected outputs

The project runs for nine months (Table 8), with a progress review at the end of month 3.

Months	Work
1	Evaluation protocol and registration of thresholds; literature update; ethics request; cluster access; extend the SEC data to the most recent quarter; DQC and EBA rule-change benchmark; rule register
2–3	RQ1 on DQC and EBA (finding and updating); dating of rule versions and models for the exposure check; start of the D4 reviews; progress review at the end of month 3
4–5	RQ2 on SEC data, both settings and the five selection methods; ESEF replication; data pipeline with quality gate and shadow mode; at the end of month 5, freeze and tag the selected rules and thresholds
6–7	RQ3: replay on real reports, clean evaluation periods and the cost of outdated rules; start of the prospective run at the beginning of month 6; completion of D4 and the alert sample
8	Prospective run continues; analysis across datasets; robustness checks; publication of the benchmark; demonstration to Avanade and Rabobank
9	Final collection of corrections for the prospective run; writing, documentation and contingency

Table 8: Nine-month plan.

The minimum scope is RQ1 on the DQC and EBA rules; RQ2 and RQ3 on SEC data, including both generation settings, the five selection methods, the clean evaluation periods and a prospective run of at least two months; at least two model families; and the double-reviewed part of D4. ESEF (D3) is an extension. If the work becomes too large, first the ESEF analysis, then the number of models and then the length of the prospective run are reduced. The held-out evaluation is not reduced. Table 9 lists the main risks.

Risk	Response
Simple methods already solve most rule updates	Report where the boundary lies; this shows a team where AI is worth using.
The experts' update is not the only correct one	Run proposals on test reports where possible and check differing proposals by hand.
Models saw the expert updates during training	Compare rule versions published before and after each model's release; check OLMo's public training data.
Rules from the gap-guided setting only learn the planted errors	Test on new companies, hold back one error type, and test the same rules on real corrections in RQ3.
Few real errors in D4	Favour reporting-error types when selecting D4, include every D4 report in the replay, and report confidence intervals.
The test over time uses information from the future	Store three dates per alert, evaluate model-written rules only after the model's release, and add a prospective run.
Older DQC rule packages do not run with current software	Filter the current rules by their first release date, as in the pilot, and report this limitation.

Table 9: Main risks and responses.

The expected outputs are a reproducible comparison of AI and non-AI rule maintenance; a public benchmark of expert rule changes; an evaluation of two ways to generate new rules and five selection methods at fixed alert budgets; a test of the resulting rules on real reports over time; a

reviewed set of real SEC corrections, subject to permissions; and a small platform that shows the procedure in use. The conclusions may show that simple methods are enough for many updates, or that generated rules add little beyond the expert rules. These are valid outcomes when they are supported by the final experiments.

References

[1] Basel Committee on Banking Supervision. Principles for effective risk data aggregation and risk reporting (BCBS 239). January 2013. https://www.bis.org/publ/bcbs239.htm.

[2] European Central Bank. Guide on effective risk data aggregation and risk reporting. May 2024. https://doi.org/10.2866/081671.

[3] European Central Bank. ECB Annual Report on supervisory activities 2024, Section 1.2.3.2: Risk data aggregation and risk reporting. 2025. https://www.bankingsupervision.europa.eu/press/other-publications/annual-report/html/ssm.ar2024~700cba1314.en.html.

[4] L. Tran. Correspondence with the author, 4 September 2026. Unpublished; observations specific to his working context.

[5] XBRL US Data Quality Committee. Rule DQC_0004: Element Values Are Equal, including the official message example. https://xbrl.us/data-rule/dqc_0004/.

[6] European Banking Authority. Single Rulebook Q&A 2022_6353, validation rule v6053_m, final answer 31 March 2023. https://eba.europa.eu/single-rule-book-qa/qna/view/publicId/2022_6353.

[7] Financial Accounting Standards Board. US GAAP Financial Reporting Taxonomy, releases 2023 to 2026, including the deprecated-concepts files. https://xbrl.fasb.org/us-gaap/.

[8] Financial Accounting Standards Board. 2026 GAAP Financial Reporting Taxonomy and Data Quality Committee Rules Taxonomy Technical Guide. 2026. https://xbrl.fasb.org/resources/annualrelease/2026/GAAP_Financial_Reporting_Taxonomy_and_Data_Quality_Committee_Rules_Taxonomy_Technical_Guide.pdf.

[9] European Banking Authority. Validation rules, rule workbooks for the reporting framework releases up to 3.5, including change labels and deactivation history. https://www.eba.europa.eu/risk-and-data-analysis/reporting/reporting-frameworks/validation-rules-packages.

[10] European Banking Authority. DPM data dictionary. https://www.eba.europa.eu/risk-and-data-analysis/reporting/dpm-data-dictionary.

[11] T. Oosterbaan. Pilot B: rule maintenance when a data standard changes. Project working report, masterthesis_direction3_pilot, docs/pilot_b_rule_maintenance.md, November 2026. Unpublished.

[12] XBRL US Data Quality Committee. DQC US rules, documentation and reference tests, version 31. Official repository, accessed November 2026. https://github.com/DataQualityCommittee/dqc_us_rules.

[13] A. Martin, E. C. de Almeida, O. Romero, and A. Queralt. How and why false denial constraints are discovered. Proceedings of the VLDB Endowment, 18(10):3477–3489, 2025. https://doi.org/10.14778/3748191.3748209.

[14] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang. An empirical study of the non-determinism of ChatGPT in code generation. ACM Transactions on Software Engineering and Methodology, 34(2), 2025. https://doi.org/10.1145/3697010.

[15] T. Oosterbaan. Pilot report: AI-written data-quality rules versus expert controls on SEC XBRL filings. Project working report, masterthesis_direction3_pilot, docs/pilot_report.md, October 2026. Unpublished; numerical results reported by the author.

[16] T. Oosterbaan. First pilot: code, configurations and results. Repository masterthesis_pilot_final, 2026. Private.

[17] T. Oosterbaan. Backtest feasibility check. Project working report, masterthesis_direction3_pilot, docs/backtest_feasibility.md, November 2026. Unpublished.

[18] T. Oosterbaan. SEC rule-generation pilot, follow-up selection test, EBA side study, rule-maintenance pilot and amendment timing: code, results and reports. Public repository https://github.com/oosterbaanvu/masterthesis_direction3_pilot, accessed 9 November 2026.

[19] P. Jung, S. Jäger, N. Chandler, and F. Biessmann. Towards realistic error models for tabular data. ACM Journal of Data and Information Quality, 17(4), 2025. https://doi.org/10.1145/3774914.

[20] US Securities and Exchange Commission. Financial Statement Data Sets, 2023q1 to 2025q2. https://www.sec.gov/data-research/sec-markets-data/financial-statement-data-sets.

[21] B. Hui et al. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024.

[22] A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025.

[23] Z. Abedjan et al. Detecting data errors: where are we and what needs to be done? Proceedings of the VLDB Endowment, 9(12):993–1004, 2016. https://doi.org/10.14778/2994509.2994518.

[24] M. Abdelaal, C. Hammacher, and H. Schöning. REIN: a comprehensive benchmark framework for data cleaning methods in ML pipelines. EDBT, 2023. https://doi.org/10.48786/edbt.2023.43.

[25] R. Alexander et al. Evaluating the decency and consistency of data validation tests generated by LLMs. arXiv:2310.01402, 2023.

[26] A. Akella, A. Kaul, K. Narayanam, and S. Mehta. Quality assessment of tabular data using large language models and code generation. EMNLP Industry Track, 2025. https://doi.org/10.18653/v1/2025.emnlp-industry.183.

[27] A.-C. Glock et al. Data quality rule generation with LLMs. arXiv:2609.06053, September 2026.

[28] J. Biggiogera et al. AuDQ: auditable data quality through agentic rule discovery. EMNLP Industry Track, 2026. https://aclanthology.org/2026.emnlp-industry.113/.

[29] T. Rehberger, T. Hütter, L. Ehrlinger, and W. Wöß. Evaluating data quality tools: measurement capabilities and LLM integration. arXiv:2604.09163, 2026.

[30] Systems and methods for automatically ensuring data quality using LLMs. US Patent 12,632,447.

[31] S. Shankar et al. SPADE: synthesizing data quality assertions for large language model pipelines. Proceedings of the VLDB Endowment, 17(12), 2024. https://doi.org/10.14778/3685800.3685835.

[32] Z. L. Liu, S. Pandit, X. Ye, E. Choi, and G. Durrett. CodeUpdateArena: benchmarking knowledge editing on API updates. arXiv:2407.06249, 2024.

[33] D. Misra, N. Islah, V. May, B. Rauby, Z. Wang, J. Gehring, A. Orvieto, M. Chaudhary, E. B. Muller, I. Rish, S. E. Kahou, and M. Caccia. GitChameleon 2.0: evaluating AI code generation against Python library version incompatibilities. arXiv:2507.12367, 2025.

[34] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: can language models resolve real-world GitHub issues? ICLR, 2024. arXiv:2310.06770.

[35] M. Volkovs, F. Chiang, J. Szlichta, and R. J. Miller. Continuous data cleaning. ICDE, 244–255, 2014. https://doi.org/10.1109/ICDE.2014.6816655.

[36] D. Tu et al. Auto-Validate by-History: auto-program data quality constraints to validate recurring data pipelines. KDD, 2023. https://doi.org/10.1145/3580305.3599776.

[37] Arelle. Open-source XBRL processor. https://arelle.org.

[38] European Securities and Markets Authority. Enforcement of corporate reporting: ESEF reporting. https://www.esma.europa.eu/issuer-disclosure/enforcement-corporate-reporting.

[39] XBRL International. ESEF Data Quality Rules. https://esef-dqr.xbrl.org.

[40] XBRL International. ESEF DQR conformance suite. https://gitlab.xbrl.org/xii/dqr-conformance.

[41] XBRL US Data Quality Committee. DQC_IFRS_0080: IFRS non-negative items, rule documentation. https://github.com/DataQualityCommittee/dqc_us_rules/blob/master/docs/ESEF/DQC_IFRS_0080/DQC_0080.md.

[42] J. Cohen. A coefficient of agreement for nominal scales. Educational and Psychological Measurement, 20(1):37–46, 1960.

[43] Team OLMo. 2 OLMo 2 Furious. arXiv:2501.00656, 2024.

[44] J. Liu et al. OLMoTrace: tracing language model outputs back to trillions of training tokens. ACL System Demonstrations, 2025. arXiv:2504.07096.

[45] University of Groningen. Hábrók cluster documentation: Apptainer containers. https://wiki.hpc.rug.nl/habrok/examples/apptainer.

[46] OpenAI. gpt-oss-120b and gpt-oss-20b model card. arXiv:2508.10925, 2025.

[47] M. Zaharia et al. Apache Spark: a unified engine for big data processing. Communications of the ACM, 59(11):56–65, 2016. https://doi.org/10.1145/2934664.